Cargo:	Docente Especialista		
Nombre:	Elvis Pachacama Cabezas		
Asignatura:	Seguridad Informática		
Carrera:	Desarrollo de Software	**Nivel:**	5to nivel
Estudiante:	Génesis Escobar		

ACTIVIDAD PRÁCTICO EXPERIMENTAL EN EL ENTORNO ACADÉMICO

Aplicación Java de cifrado y descifrado afín

Producto final: aplicación de escritorio funcional, proyecto fuente e informe de evidencias.

Objetivos

Desarrollar una aplicación gráfica en Java que cifre y descifre textos mediante la implementación manual del cifrado afín, utilizando aritmética modular, validaciones y pruebas, para comprender la transformación de la información y las limitaciones de un cifrado clásico.

Antecedentes/Escenario

Un laboratorio académico necesita una herramienta para demostrar cómo una clave transforma un mensaje y permite recuperarlo. El usuario debe escribir un texto en una caja de texto, ingresar dos valores de clave y observar el resultado de cifrar o descifrar.

El cifrado afín pertenece a los cifrados clásicos de sustitución: asigna a cada letra un número y aplica una transformación matemática invertible. Esta práctica desarrolla comprensión del algoritmo; no constituye una solución para proteger datos reales (Menezes et al., 1996).

Recursos necesarios

Computadora con JDK 17 o superior; IntelliJ IDEA, NetBeans, Eclipse o un editor equivalente; biblioteca estándar Java Swing para la interfaz; cuaderno o procesador de textos para los cálculos; esta guía y un compresor ZIP. No se requieren servicios en línea ni dependencias externas para ejecutar la aplicación.

Planteamiento del problema

Implemente un programa de escritorio que reciba texto, aplique el cifrado afín y permita recuperar el mensaje utilizando las mismas claves. El estudiante debe programar el recorrido de caracteres, el máximo común divisor, el inverso modular y las operaciones de cifrado y descifrado.

La interfaz y la lógica deben estar separadas. No se permite delegar el algoritmo a una biblioteca criptográfica, una API o un servicio externo. La organización del trabajo, el plazo y la plataforma de entrega serán establecidos por el docente.

Pasos por realizar

Paso 1 Comprender el alfabeto y las claves

Trabaje con 26 letras: A = 0, B = 1, C = 2, …, Z = 25. Aplique la misma transformación a las minúsculas, conservando su forma. Mantenga sin cambios espacios, saltos de línea, números, signos, Ñ, ñ y letras con tilde. No elimine caracteres ni convierta todo el texto a mayúsculas.

La clave tiene dos valores enteros, a y b. El valor a debe estar entre 1 y 25 y cumplir MCD(a, 26) = 1. Sus valores válidos son 1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23 y 25. El valor b debe estar entre 0 y 25. Use a = 5 y b = 8 como valores iniciales.

Paso 2 Interpretar las funciones matemáticas

Para cifrar, M representa la posición de una letra y C la posición resultante. La operación mod 26 devuelve un valor entre 0 y 25:

$$C = (a \times M + b) \, mod \, 26$$

Para descifrar, calcule primero el inverso modular de a, es decir, un entero que multiplicado por a deje residuo 1 al dividir entre 26:

$$a \times a^{-1} \equiv 1 \, (mod \, 26)$$

$$M = (a^{-1} \times (C - b)) \, mod \, 26$$

El inverso modular no es la división 1/a. Puede encontrarlo probando candidatos del 1 al 25. Para a = 5, el inverso es 21, porque 5 × 21 = 105 y 105 mod 26 = 1.

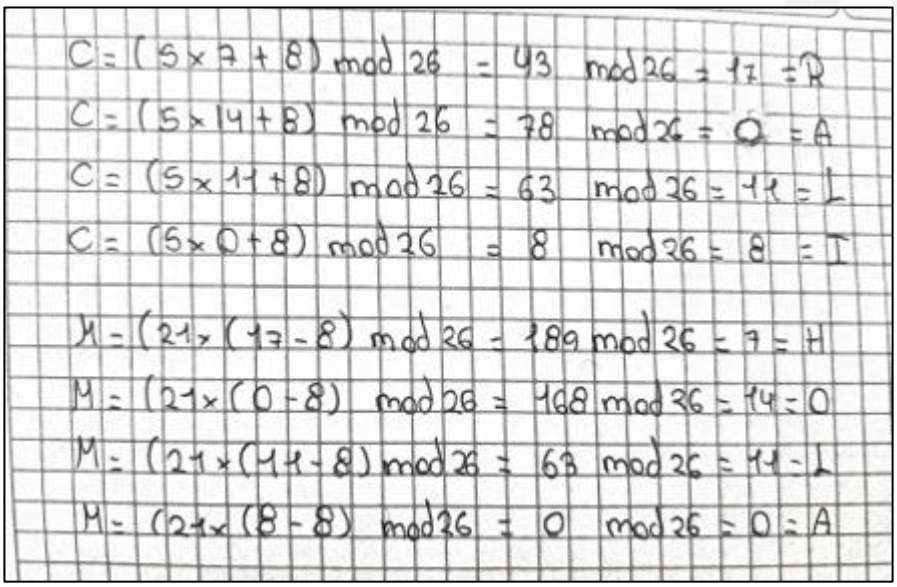
Paso 3 Resolver una transformación a mano

Ejemplo de elaboración docente: con a = 5, b = 8 y H = 7, el cifrado y su comprobación son:

$$C = (5 \times 7 + 8) \, mod \, 26 = 43 \, mod \, 26 = 17 = R$$

$$M = 21 \times (17 - 8) \, mod \, 26 = 189 \, mod \, 26 = 7 = H$$

Antes de programar, cifre manualmente la palabra HOLA y descifre su resultado. Registre las posiciones, la sustitución de valores y los residuos. Su programa deberá producir HOLA MUNDO → RALI QEVXA y recuperar exactamente el texto original.



Paso 4 Normalizar los residuos negativos

En Java, el operador % puede producir un residuo negativo. Defina una función que transforme el resultado en una posición válida del alfabeto:

$$mod26(n) = ((n \,\%\, 26) + 26) \,\%\, 26$$

Compruebe el caso de descifrar A con las claves iniciales: el cálculo utiliza 21 × (0 − 8) = −168. Su normalización debe producir 14, correspondiente a O.

Pasos por realizar

Paso 5 Implementar la lógica del cifrado

Cree una clase CifradoAfin y programe métodos propios para validar claves, calcular el MCD, encontrar el inverso modular, cifrar y descifrar. Métodos sugeridos: mcd, inversoModular, cifrar y descifrar. Puede añadir auxiliares para normalizar residuos y transformar caracteres.

Recorra el texto carácter por carácter. Para una letra válida, obtenga su posición, aplique la fórmula y reconstruya la letra conservando mayúscula o minúscula. Copie los demás caracteres sin modificarlos. Use operaciones aritméticas, condicionales, ciclos y manejo de cadenas.

No utilice javax.crypto, Cipher, bibliotecas de terceros ni servicios que realicen cifrado. Tampoco utilice BigInteger.modInverse o un método de biblioteca para calcular el MCD: esas operaciones forman parte del trabajo. Base64 es una codificación y no cumple el algoritmo solicitado. Se permite la biblioteca estándar para interfaz, eventos y manipulación de texto.

1. Clase de Cifrado a fin con las variables para almacenar el alfabeto y modulo 27.

```
public class CifradoAfin {

    //variable para almacenar los 27 caracteres del alfabeto incluyendo
la ñ
    private static final String ALFABETO = "abcdefghijklmnñopqrstuvwxyz";
    //variable para indicar el modulo de la operacion, que es 27 por los
27 caracteres del alfabetos
    private static final int MODULO = 27;
```

2. Método para normalizar residuos negativos.

```
//1. normalizar residuos negativos
    //como el operador % en Java puede dar resultados negativos, este
método asegura que siempre obtengamos un resultado positivo dentro del
rango de 0 a 26.
    public int mod27(int n) {
        return ((n % MODULO)+ MODULO) % MODULO;
    }
```

3. Método para el máximo común divisor.

```
    //2. máximo común divisor
    //algoritmo de Euclides para calcular el máximo común divisor (MCD)
de dos números enteros. Se utiliza para verificar si la clave 'a' es
coprima con 27, lo cual es un requisito para que el cifrado y descifrado
funcionen correctamente.
    public int mcd(int a, int b) {
        //math abs elimina el signo negativo de los numeros, para que no
afecte el calculo del mcd
        a= Math.abs(a);
        b= Math.abs(b);
```

```
        //ciclo hasta que el residuo sea 0, lo que indica que hemos
encontrado el MCD
        while (b!=0) {
            int residuo= a % b; //calcular el residuo de a entre b
            a=b; //el divisor, es decir, b, pasa a ser el nuevo a
            b=residuo; //el residuo pasa a ser el nuevo b para la
siguiente iteración
        }
        return a;
    }
```

4. Método para el inverso modular.

```
    //3. inverso modular para descifrar el texto cifrado
    public int inversoModular (int a) {
        //a tiene que estar en el rango de 0 a 26 para que el ciclo
funcione correctamente
        a= mod27(a);

        //probamos numeros del 1 al 26
        for(int x=1; x < 27; x++) {
            //multiplicamos a por x y verificamos si el residuo de la
division entre 27 es igual a 1
            if ((a * x) % 27 == 1) {
                return x; //almacena el valor de x que cumple la
condición y lo retorna
            }
        }
        //si el ciclo termina y no encuentra un inverso modular de a,
retorna -1 indicando que no existe un inverso modular para el valor dado
de a.
        return -1;
    }
```

5. Método para cifrar.

```
//4. metodo para cifrar
    public String cifrar(String texto, int a, int b) {
        //stringbuilder me permite trabajar en la misma cadena de texto
sin llenar los espacios de memoria
        StringBuilder resultado = new StringBuilder();

        //valida el rango de a y b y que a sea coprima de 27; si no,
lanza una excepcion
        validarClaves(a, b);

        //recorrer el texto
        for(int i=0; i< texto.length(); i++) {
            //obtiene el caracter en la posicion i
```

```
            char c = texto.charAt(i);

            // Detectamos si es mayúscula y la pasamos a minúscula
temporalmente
            boolean esMayuscula = Character.isUpperCase(c);
            char cMinuscula = Character.toLowerCase(c);

            //obtiene el indice del caracter
            int index = ALFABETO.indexOf(cMinuscula);

            if (index != -1) { //vemos que el indice este en el alfabeto
de 27
                //FORMULA: c=(a*m+b) mod 27
                int nuevoIndice = mod27((a*index)+b);
                char nuevoChar = ALFABETO.charAt(nuevoIndice);

                // Si era mayúscula originalmente, la devolvemos como
mayúscula
                resultado.append(esMayuscula ?
Character.toUpperCase(nuevoChar) : nuevoChar);

            }else{
                //resultado .append(c) conserva los caracteres que no
están en el alfabeto, como espacios, signos de puntuación y números, sin
cambios.
                resultado.append(c);
            }
        }
        //convierte el StringBuilder a String y lo retorna como resultado
final del cifrado
        return resultado.toString();
    }
```

6. Método para descifrar

```
// 5. metodo para descifrar
    public String descifrar(String texto, int a, int b) {
        //stringbuilder me permite trabajar en la misma cadena de texto
sin llenar los espacios de memoria
        StringBuilder resultado = new StringBuilder();

        //valida el rango de a y b y que a sea coprima de 27; si no,
lanza una excepcion
        validarClaves(a, b);

        //obtener inverso modular (a^-1)
        int aInverso = inversoModular(a);

        //recorrer el texto
```

```
        for (int i=0; i<texto.length(); i++) {
            char c = texto.charAt(i);
            boolean esMayuscula = Character.isUpperCase(c);
            char cMinuscula = Character.toLowerCase(c);
            int index = ALFABETO.indexOf(cMinuscula);

            if(index !=-1) {
                // formula: M = (a^-1 * (C - b)) mod 27
                //multiplicamos el inverso por la resta de la posición
(C) menos el desplazamiento (b)
                int operacion = aInverso * (index-b);

                // mod27 arregla automáticamente si operacion da un
numero negativo
                int nuevoIndice = mod27(operacion);
                char nuevoChar = ALFABETO.charAt(nuevoIndice);

                 resultado.append(esMayuscula ?
Character.toUpperCase(nuevoChar) : nuevoChar);
            }else{
                //conservamos espacios, signos y numeros sin cambio
                resultado.append(c);
            }
        }
        return resultado.toString();
    }
```

7. Método para validar las claves.

```
// 6. validar claves
    //a debe estar entre 1 y 26 y ser coprima de 27 (para que exista el
inverso modular); b debe estar entre 0 y 26
    private void validarClaves(int a, int b) {
        if (a < 1 || a >= MODULO) {
            throw new IllegalArgumentException("La clave a debe estar
entre 1 y 26.");
        }
        if (mcd(a, MODULO) != 1) {
            throw new IllegalArgumentException("La clave a no es coprima
de 27.");
        }
        if (b < 0 || b >= MODULO) {
            throw new IllegalArgumentException("La clave b debe estar
entre 0 y 26.");
        }
    }
}
```

Paso 6 Construir la interfaz gráfica con Swing

Incluya una ventana con título, etiquetas claras y los siguientes componentes:

- Área de entrada editable y multilínea, con desplazamiento.
- Área de resultado multilínea de solo lectura.
- Controles para ingresar los enteros a y b.
- Botones Cifrar, Descifrar, Usar resultado como entrada y Limpiar.
- Un mensaje de estado o diálogo para informar resultados y errores.

JTextArea permite trabajar con texto multilínea y puede colocarse en un JScrollPane para añadir desplazamiento (Oracle, s. f.-a). Organice la ventana con un administrador de diseño. Inicie la interfaz mediante SwingUtilities.invokeLater y gestione las acciones de los botones mediante eventos (Oracle, s. f.-b).

```
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;

public class CifradoAfinGUI extends JFrame {

    private CifradoAfin logica;

    private JTextArea txtEntrada, txtSalida;
    private JTextField txtClaveA, txtClaveB;
    private JButton btnCifrar, btnDescifrar, btnLimpiar,
btnUsarResultado;

    // Paleta de colores MODO OSCURO (Suave a la vista)
    private final Color FONDO_PRINCIPAL = new Color(30, 32, 36);     //
Gris muy oscuro
    private final Color FONDO_PANELES = new Color(43, 47, 54);       //
Gris un poco más claro para "tarjetas"
    private final Color COLOR_TEXTO = new Color(230, 234, 240);      //
Blanco hueso/azulado
    private final Color COLOR_TEXTO_SECUNDARIO = new Color(170, 175,
185); // Gris claro
    private final Color COLOR_BORDE = new Color(64, 68, 75);         //
Borde sutil

    // Colores de botones vibrantes para contrastar
    private final Color BTN_PRIMARIO = new Color(88, 101, 242);      //
Azul moderno (estilo Discord)
    private final Color BTN_SECUNDARIO = new Color(79, 84, 92);      //
Gris para limpiar
    private final Color BTN_EXITO = new Color(46, 204, 113);         //
Verde esmeralda para descifrar

    // Fuentes modernas y escaladas
```

```
    private final Font FUENTE_TITULO = new Font("Segoe UI", Font.BOLD,
24);
    private final Font FUENTE_SUBTITULO = new Font("Segoe UI", Font.BOLD,
16);
    private final Font FUENTE_BASE = new Font("Segoe UI", Font.PLAIN,
15);
    private final Font FUENTE_CLAVES = new Font("Segoe UI", Font.BOLD,
22); // Claves más grandes

    public CifradoAfinGUI() {
        logica = new CifradoAfin();
        configurarVentana();
        inicializarComponentes();
    }

    private void configurarVentana() {
        setTitle("Cifrado Afín - Workspace");
        setSize(950, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        getContentPane().setBackground(FONDO_PRINCIPAL);
    }

    private void inicializarComponentes() {
        // --- PANEL SUPERIOR (TÍTULO) ---
        JPanel panelTitulo = new JPanel();
        panelTitulo.setBackground(FONDO_PRINCIPAL);
        panelTitulo.setBorder(new EmptyBorder(20, 20, 10, 20));
        JLabel lblTitulo = new JLabel("Sistema de Cifrado Afín");
        lblTitulo.setForeground(COLOR_TEXTO);
        lblTitulo.setFont(FUENTE_TITULO);
        panelTitulo.add(lblTitulo);
        add(panelTitulo, BorderLayout.NORTH);

        // --- PANEL CENTRAL (DIVIDIDO EN DOS) ---
        // Usamos GridLayout(1, 2) para dividir la pantalla a la mitad
        JPanel panelSplit = new JPanel(new GridLayout(1, 2, 20, 0));
        panelSplit.setBackground(FONDO_PRINCIPAL);
        panelSplit.setBorder(new EmptyBorder(10, 25, 25, 25));

        // 1. CONSTRUIR LADO IZQUIERDO (Claves y Botones)
        panelSplit.add(construirPanelIzquierdo());

        // 2. CONSTRUIR LADO DERECHO (Áreas de Texto)
        panelSplit.add(construirPanelDerecho());

        add(panelSplit, BorderLayout.CENTER);
```

```
        // --- EVENTOS ---
        btnCifrar.addActionListener(e -> procesarTexto(true));
        btnDescifrar.addActionListener(e -> procesarTexto(false));

        btnUsarResultado.addActionListener(e -> {
            if (!txtSalida.getText().isEmpty()) {
                txtEntrada.setText(txtSalida.getText());
                txtSalida.setText("");
            }
        });

        btnLimpiar.addActionListener(e -> {
            txtEntrada.setText("");
            txtSalida.setText("");
        });
    }

    // --- CONSTRUCTOR DEL LADO IZQUIERDO ---
    private JPanel construirPanelIzquierdo() {
        JPanel panelIzq = new JPanel(new BorderLayout(0, 20));
        panelIzq.setBackground(FONDO_PANELES);
        panelIzq.setBorder(BorderFactory.createCompoundBorder(
            new LineBorder(COLOR_BORDE, 1, true),
            new EmptyBorder(25, 25, 25, 25)
        ));

        // Título de la sección
        JLabel lblConfig = new JLabel("Configuración y Controles",
SwingConstants.CENTER);
        lblConfig.setFont(FUENTE_SUBTITULO);
        lblConfig.setForeground(COLOR_TEXTO);
        panelIzq.add(lblConfig, BorderLayout.NORTH);

        // Zona de Claves (GridBagLayout para centrarlas y hacerlas
grandes)
        JPanel panelClaves = new JPanel(new GridLayout(2, 2, 15, 20));
        panelClaves.setBackground(FONDO_PANELES);
        panelClaves.setBorder(new EmptyBorder(20, 20, 30, 20));

        JLabel lblA = new JLabel("Clave 'a':", SwingConstants.RIGHT);
        lblA.setFont(FUENTE_SUBTITULO);
        lblA.setForeground(COLOR_TEXTO_SECUNDARIO);
        txtClaveA = crearCampoClave();

        JLabel lblB = new JLabel("Clave 'b':", SwingConstants.RIGHT);
        lblB.setFont(FUENTE_SUBTITULO);
        lblB.setForeground(COLOR_TEXTO_SECUNDARIO);
        txtClaveB = crearCampoClave();
```

```
        panelClaves.add(lblA);
        panelClaves.add(txtClaveA);
        panelClaves.add(lblB);
        panelClaves.add(txtClaveB);

        panelIzq.add(panelClaves, BorderLayout.CENTER);

        // Zona de Botones (Cuadrícula 2x2)
        JPanel panelBotones = new JPanel(new GridLayout(2, 2, 15, 15));
        panelBotones.setBackground(FONDO_PANELES);

        btnCifrar = crearBoton("Cifrar", BTN_PRIMARIO, Color.WHITE);
        btnDescifrar = crearBoton("Descifrar", BTN_EXITO, Color.WHITE);
        btnUsarResultado = crearBoton("Usar Resultado", BTN_PRIMARIO,
Color.WHITE);
        btnLimpiar = crearBoton("Limpiar", BTN_SECUNDARIO, Color.WHITE);

        panelBotones.add(btnCifrar);
        panelBotones.add(btnDescifrar);
        panelBotones.add(btnUsarResultado);
        panelBotones.add(btnLimpiar);

        panelIzq.add(panelBotones, BorderLayout.SOUTH);

        return panelIzq;
    }

    // --- CONSTRUCTOR DEL LADO DERECHO ---
    private JPanel construirPanelDerecho() {
        JPanel panelDer = new JPanel(new GridLayout(2, 1, 0, 15));
        panelDer.setBackground(FONDO_PRINCIPAL);

        txtEntrada = crearAreaTexto("Escribe aquí el texto...");
        txtSalida = crearAreaTexto("");
        txtSalida.setEditable(false);
        txtSalida.setBackground(new Color(36, 40, 46)); // Un poco más
oscuro para modo lectura

        panelDer.add(crearContenedorTexto("Texto de Entrada:",
txtEntrada));
        panelDer.add(crearContenedorTexto("Resultado Generado:",
txtSalida));

        return panelDer;
    }

    // --- MÉTODOS AUXILIARES DE DISEÑO ---

    private JTextArea crearAreaTexto(String placeholder) {
```

```
        JTextArea area = new JTextArea();
        area.setFont(FUENTE_BASE);
        area.setForeground(COLOR_TEXTO);
        area.setBackground(FONDO_PANELES);
        area.setCaretColor(Color.WHITE); // Cursor visible en modo oscuro
        area.setLineWrap(true);
        area.setWrapStyleWord(true);
        area.setMargin(new Insets(12, 12, 12, 12));
        return area;
    }

    private JTextField crearCampoClave() {
        JTextField campo = new JTextField();
        campo.setFont(FUENTE_CLAVES); // Fuente mucho más grande
        campo.setForeground(Color.WHITE);
        campo.setBackground(FONDO_PRINCIPAL);
        campo.setCaretColor(Color.WHITE);
        campo.setHorizontalAlignment(JTextField.CENTER);
        campo.setBorder(BorderFactory.createCompoundBorder(
            new LineBorder(COLOR_BORDE, 2),
            new EmptyBorder(10, 10, 10, 10)
        ));
        return campo;
    }

    private JPanel crearContenedorTexto(String titulo, JTextArea
areaTexto) {
        JPanel panel = new JPanel(new BorderLayout(0, 8));
        panel.setBackground(FONDO_PRINCIPAL);

        JLabel label = new JLabel(titulo);
        label.setFont(FUENTE_SUBTITULO);
        label.setForeground(COLOR_TEXTO_SECUNDARIO);

        JScrollPane scroll = new JScrollPane(areaTexto);
        scroll.setBorder(new LineBorder(COLOR_BORDE, 1, true));
        scroll.getVerticalScrollBar().setBackground(FONDO_PRINCIPAL);

        panel.add(label, BorderLayout.NORTH);
        panel.add(scroll, BorderLayout.CENTER);
        return panel;
    }

    private JButton crearBoton(String texto, Color fondoNormal, Color
textoColor) {
        JButton boton = new JButton(texto);
        boton.setBackground(fondoNormal);
        boton.setForeground(textoColor);
        boton.setFocusPainted(false);
```

```
        boton.setFont(new Font("Segoe UI", Font.BOLD, 14));
        boton.setCursor(new Cursor(Cursor.HAND_CURSOR));
        boton.setBorder(new EmptyBorder(12, 15, 12, 15));

        // Evita que Windows sobreescriba el diseño
        boton.setContentAreaFilled(false);
        boton.setOpaque(true);

        // Colores para las animaciones del botón
        Color fondoHover = fondoNormal.brighter();
        Color fondoPresionado = fondoNormal.darker();

        // EFECTOS DE INTERACCIÓN (Hover y Click)
        boton.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                boton.setBackground(fondoHover);
            }
            @Override
            public void mouseExited(MouseEvent e) {
                boton.setBackground(fondoNormal);
            }
            @Override
            public void mousePressed(MouseEvent e) {
                boton.setBackground(fondoPresionado);
            }
            @Override
            public void mouseReleased(MouseEvent e) {
                boton.setBackground(fondoHover);
            }
        });

        return boton;
    }

    // --- LÓGICA ---
    private void procesarTexto(boolean esCifrado) {
        try {
            //se borra el resultado anterior para no mostrarlo como
actual si la operacion falla
            txtSalida.setText("");
            String texto = txtEntrada.getText();
            if (texto.isEmpty()) {
                JOptionPane.showMessageDialog(this, "El área de entrada
está vacía.", "Advertencia", JOptionPane.WARNING_MESSAGE);
                return;
            }
            int a = Integer.parseInt(txtClaveA.getText().trim());
            int b = Integer.parseInt(txtClaveB.getText().trim());
```

```
            String resultado = esCifrado ? logica.cifrar(texto, a, b) :
logica.descifrar(texto, a, b);
            txtSalida.setText(resultado);

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Las claves 'a' y 'b'
deben ser números enteros.", "Error de Entrada",
JOptionPane.ERROR_MESSAGE);
        } catch (IllegalArgumentException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Clave
Inválida", JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        try {
            // Aplicar estilo base del sistema para heredar ciertas
configuraciones nativas
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassN
ame());
        } catch (Exception e) {
            e.printStackTrace();
        }
        // sirve para ejecutar los componentes de la interfaz uno por uno
y no todos al mismo tiempo, evitando errores de renderizado
        SwingUtilities.invokeLater(() -> {
            CifradoAfinGUI ventana = new CifradoAfinGUI();
            ventana.setVisible(true);
        });
    }
}
```



Paso 7 Conectar los controles y validar

Cifrar procesa siempre el texto del área de entrada y escribe en el resultado. Descifrar procesa también el área de entrada, usando las mismas claves. Usar resultado como entrada copia el resultado al área editable para ejecutar la operación inversa. Limpiar vacía ambas áreas y conserva las claves.

Rechace una entrada vacía, claves no enteras, valores fuera de rango y un valor a sin inverso. Muestre un mensaje comprensible y permita corregir los datos sin cerrar la ventana. Antes de procesar, confirme también lo que el usuario haya escrito en los controles numéricos.

No incorpore una validación que asegure que un texto está cifrado: este algoritmo transforma cualquier secuencia de letras y no puede reconocer de forma confiable una clave incorrecta. Evite mostrar como actual un resultado anterior cuando una operación falle.





Paso 8 Comprobar y explicar el resultado

Ejecute los casos de la página siguiente. Para cada uno, registre resultado esperado, resultado obtenido y estado de la prueba. Corrija los errores y repita los casos afectados. Durante la defensa, demuestre el proceso completo con un mensaje indicado por el docente.

Desarrollo

Complete esta sección en su informe de ejecución. Debe explicar cómo implementó la actividad e incorporar resultados propios. Utilice al menos los siguientes casos de prueba; puede añadir otros.

N.º	Datos de entrada	Acción	Resultado esperado	Resultado Obtenido	Acción
1	HOLA MUNDO; a=5, b=8	Cifrar	RALI QEVXA *Con alfabeto de 27 letras: PCJI ÑFSWC*	PCJI ÑFSWC	Aprobado
2	RALI QEVXA; a=5, b=8 *Con 27 letras la entrada es PCJI ÑFSWC (salida del caso 1)*	Descifrar	HOLA MUNDO *Con 27 letras: HOLA MUNDO*	HOLA MUNDO	Aprobado
3	Hola, mundo!; a=5, b=8	Cifrar	Rali, qevxa! *Con 27 letras: Pcji, ñfswc!*	Pcji, ñfswc!	Aprobado
4	Ññ áéíóú 123!?; a=5, b=8 *Entrada exacta de la captura: Ññ áéíóú 123!?;*	Cifrar	Los caracteres se conservan sin cambios. *Con 27 letras: Ñ y ñ pertenecen al alfabeto y se cifran (Xx); tildes, números y signos se conservan.*	Xx áéíóú 123!?;	Aprobado
5	HOLA y MUNDO en líneas distintas; a=5, b=8	Cifrar	RALI y QEVXA, conservando el salto de línea. *Con 27 letras: PCJI y ÑFSWC, conservando el salto de línea.*	Pcji Ñfswc	Aprobado
6	Área de entrada vacía	Cualquiera	Mensaje de validación, sin cerrar la aplicación.	Advertencia: “El área de entrada está vacía.”	Aprobado
7	ABC; a=2 o a=13, b=8 *Valor usado en la captura: a=3*	Cifrar	Rechazar a porque MCD(a,26) no es 1. *Con módulo 27: se rechaza a=3 (MCD(3,27)=3). a=2 y a=13 sí son válidos.*	Advertencia: “La clave a no es coprima de 27”	Aprobado
8	ABC; a=5, b=26 o una clave no entera	Cifrar	Rechazar el valor o impedir ingresarlo. *Con módulo 27: b=26 es válido (rango 0–26); se rechaza una clave vacía, no entera o fuera de rango (b=27, b=−1).*	Advertencia: “La claves ‘a’ y ‘b’ deben ser números enteros.”	Aprobado

N.º	Datos de entrada	Acción	Resultado esperado	Resultado Obtenido	Acción
9	A; a=5, b=8 *Valor usado en la captura: a=5, b=8*	Descifrar	O, con manejo correcto del residuo negativo. *Con 27 letras: T (−88 mod 27 = 20). La O corresponde al alfabeto de 26 letras.*	T	Aprobado
10	ABC xyz; a=1, b=0	Cifrar	ABC xyz. La clave identidad es matemáticamente válida.	ABC xyz	Aprobado
11	Texto propio, con mayúsculas y signos *Texto: Hola me llamo Génesis!; a=5, b=9*	Ida y vuelta	Descifrar(cifrar(texto,a,b),a,b) recupera el original.	“Hola me llamo Génesis!” Qdkj oc kkjod Métcwvw!	Aprobado

1. **Caso 1:** El caso de prueba esta hecho con un abecedario de 26 caracteres, el nuevo pedido por el docente debia incluir la letra ñ por lo tanto las letras cambian de posición, el resultado obtenido es correcto. Ejemplo con la H: posición 7, (5 × 7 + 8) mod 27 = 43 mod 27 = 16, que corresponde a P. Con 26 letras daba R; con 27 letras cambia porque la Ñ ocupa la posición 14 y desplaza a las letras siguientes. El programa devolvió PCJI ÑFSWC, que coincide con el cálculo manual hecho con módulo 27.



2. **Caso 2:** Se tomó como entrada el resultado del caso 1 (PCJI ÑFSWC) y se descifró con las mismas claves, a=5 y b=8. El inverso modular de 5 en módulo 27 es 11, porque 5 × 11 = 55 = 2 × 27 + 1. El programa recuperó exactamente HOLA MUNDO, lo que confirma que descifrar deshace a cifrar. La entrada RALI QEVXA de la guía pertenece al alfabeto de 26 letras; con 27 letras se descifraría como CTGA RKSÑT.



3. **Caso 3:** Se conservan la mayúscula inicial, la coma, el espacio y el signo de admiración; solo se transforman las letras: H→P, o→c, l→j, a→i (Pcji) y m→ñ, u→f, n→s, d→w, o→c (ñfswc). La salida es la del caso 1 pero respetando mayúsculas y minúsculas, por lo que la conversión mayúscula/minúscula del método cifrar funciona correctamente.



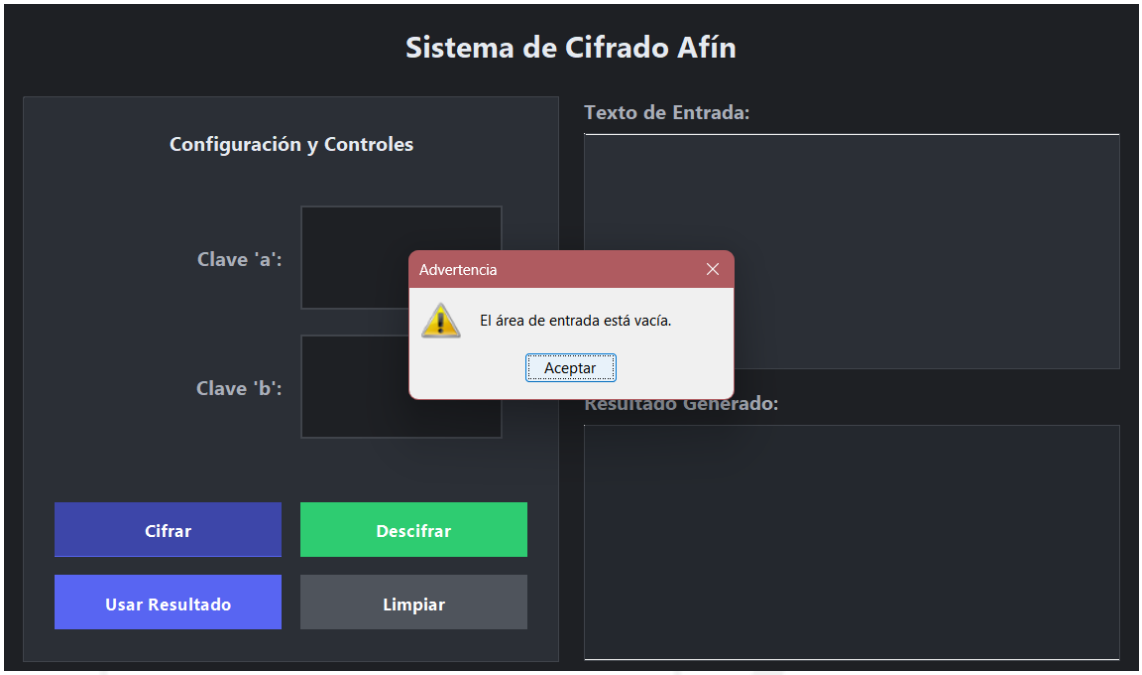
4. **Caso 4:** Ñ y ñ están en el alfabeto de 27 letras (posición 14), por eso se cifran como X y x: (5 × 14 + 8) mod 27 = 78 mod 27 = 24, que es la x. Las vocales con tilde (á, é, í, ó, ú), los números y los signos no están en el alfabeto y se copian sin cambios. El punto y coma final de la captura forma parte del texto que se ingresó. Esto difiere de lo que esperaba la guía (con 26 letras la Ñ se conservaba) y es consecuencia de haber incluido la ñ en el alfabeto; el resultado es correcto para la implementación.



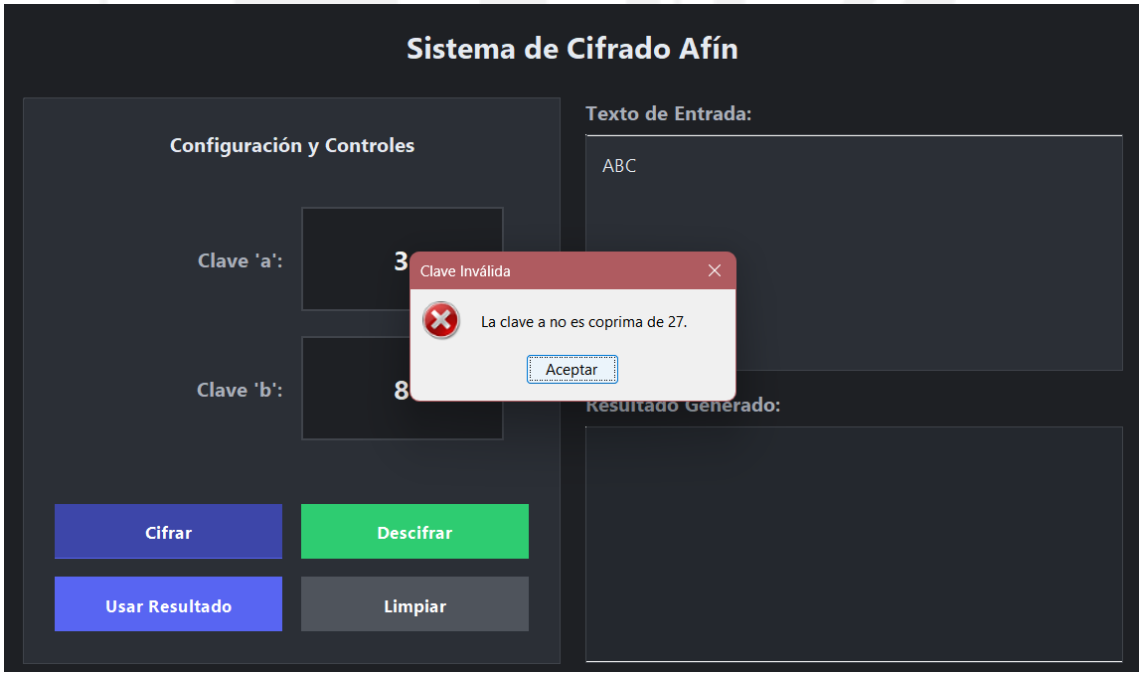
5. **Caso 5:** El salto de línea (\n) no pertenece al alfabeto, así que se copia tal cual. La primera línea HOLA se cifra como PCJI y la segunda, MUNDO, como ÑFSWC, igual que en el caso 1.



6. **Caso 6:** Con el área de entrada vacía, el método procesarTexto detecta texto.isEmpty(), muestra una advertencia con JOptionPane y termina con return antes de leer las claves o llamar a la lógica. La ventana sigue abierta y el usuario puede escribir un texto y volver a intentar.



7. **Caso 7:** Se probó a = 3. Como MCD(3, 27) = 3 y no 1, el método cifrar lanza IllegalArgumentException y la interfaz muestra «La clave a no es coprima de 27.» en el cuadro «Clave Inválida». Los valores a = 2 y a = 13 de la guía eran inválidos con módulo 26, pero con módulo 27 sí son válidos porque MCD(2, 27) = MCD(13, 27) = 1 (con a = 2, b = 8, ABC se cifra como IKM). Con módulo 27 las claves rechazadas son los múltiplos de 3: 3, 6, 9, 12, 15, 18, 21 y 24.



8. **Caso 8:** Cuando una clave está vacía o no es un número entero, Integer.parseInt lanza NumberFormatException y la interfaz muestra «Las claves 'a' y 'b' deben ser números enteros.» sin cerrarse. El texto ZEJ que se ve en la captura es de una operación anterior (ABC con a = 5 y b = 26), válida porque con módulo 27 b puede ir de 0 a 26; se vio porque esa versión del programa no borraba el resultado previo. En la versión final,

procesarTexto limpia el área de resultado antes de validar, y validarClaves rechaza además valores fuera de rango (a fuera de 1–26, b fuera de 0–26).



9. **Caso 9:** Con a = 5 y b = 8 el inverso de a es 11, así que M = 11 × (0 − 8) = −88. En Java −88 % 27 da −7 (residuo negativo); mod27 le suma 27 y obtiene 20, que corresponde a la T. El resultado correcto de descifrar A es entonces T (con 26 letras era O, como indica la guía).



10. **Caso 10:** Con a = 1 y b = 0 la función es C = (1 × M + 0) mod 27 = M, es decir, la identidad, y el texto de salida es igual al de entrada: ABC xyz. Es una clave válida porque MCD(1, 27) = 1 y su inverso es 1. La prueba también muestra que se conservan mayúsculas, minúsculas y espacios.



11. **Caso 11:** Se cifró «Hola me llamo Génesis!» con a = 5 y b = 9 y se obtuvo «Qdkj oc kkjod Métcwvw!». La é con tilde y el signo de admiración no cambian. Luego se copió el resultado a la entrada y se descifró con las mismas claves, recuperando exactamente «Hola me llamo Génesis!». Además, al ejecutar la clase CifradoAfin con las 486 combinaciones de claves válidas (18 valores de a por 27 de b), descifrar(cifrar(texto)) devolvió siempre el texto original.





Evidencias y análisis del desarrollo

Incluya el cálculo manual de HOLA, una explicación de cada método y las validaciones realizadas. Incorpore capturas legibles del cifrado, del descifrado y de al menos un error de clave. En la tabla de pruebas de su informe, añada columnas para resultado obtenido y aprobación o fallo.

Explique por qué debe conservarse la misma clave, qué ocurre con caracteres fuera de A–Z, por qué a debe ser coprimo con 26 y cómo resolvió los residuos negativos. Analice por qué este algoritmo no ofrece seguridad suficiente para proteger información real.

Cierre con tres conclusiones vinculadas al objetivo: implementación matemática, funcionamiento de la aplicación y limitaciones del mecanismo. Identifique fuentes o apoyos utilizados y cite en APA 7 la información externa.

1. Decisión de diseño: alfabeto de 27 letras

La guía plantea 26 letras (A–Z, módulo 26), pero la aplicación se implementó con 27 letras (a–z más la ñ), por lo que MODULO = 27. Esto cambia varios resultados de la guía:

- Las posiciones desde la ñ se desplazan: ñ = 14, o = 15, …, z = 26.
- HOLA MUNDO con a = 5 y b = 8 se cifra como PCJI ÑFSWC y no como RALI QEVXA.
- La clave a debe cumplir MCD(a, 27) = 1, lo que deja 18 valores válidos (1, 2, 4, 5, 7, 8, 10, 11, 13, 14, 16, 17, 19, 20, 22, 23, 25 y 26). La clave b va de 0 a 26.
- El inverso modular de 5 es 11 ($5 \times 11 = 55 = 2 \times 27 + 1$) y no 21. Por eso descifrar A con a = 5 y b = 8 da T y no O.

La hoja con el cálculo a mano (Paso 3) corresponde al alfabeto de 26 letras de la guía. A continuación se repite el cálculo de HOLA con el alfabeto de 27 letras que usa el programa.

2. Cálculo manual de HOLA (a = 5, b = 8, módulo 27)

Cifrado: $C = (a \times M + b) \bmod 27$. Descifrado: $M = a^{-1} \times (C - b) \bmod 27$, con $a^{-1} = 11$.

Letra	M	C = (5 × M + 8) mod 27	Cifrada	M = 11 × (C − 8) mod 27	Valor	Letra
H	7	43 mod 27 = 16	P	11 × 8 = 88 → 88 mod 27	7	H
O	15	83 mod 27 = 2	C	11 × (−6) = −66 → −66 mod 27	15	O
L	11	63 mod 27 = 9	J	11 × 1 = 11 → 11 mod 27	11	L
A	0	8 mod 27 = 8	I	11 × 0 = 0 → 0 mod 27	0	A

HOLA se cifra como PCJI y al descifrar se recupera HOLA. Para el residuo negativo de la O: −66 + 3 × 27 = 15.

3. Explicación de cada método

- **mod27(n):** devuelve ((n % 27) + 27) % 27. El operador % de Java puede dar un residuo negativo; por ejemplo −88 % 27 = −7. Al sumar 27 y volver a aplicar % se obtiene un valor entre 0 y 26 (en el ejemplo, 20).
- **mcd(a, b):** implementa el algoritmo de Euclides. Toma el valor absoluto de ambos números y repite mientras b sea distinto de 0: calcula el residuo de a entre b, pasa b a a y el residuo a b. Al terminar, a es el MCD. Ejemplos: mcd(5, 27) = 1 y mcd(3, 27) = 3.
- **inversoModular(a):** normaliza a con mod27 y prueba los valores x de 1 a 26 hasta encontrar uno que cumpla (a × x) % 27 == 1. Si no existe devuelve −1. Para a = 5 devuelve 11. No se usa BigInteger.modInverse.
- **validarClaves(a, b):** comprueba que a esté entre 1 y 26, que MCD(a, 27) = 1 y que b esté entre 0 y 26; si algo falla, lanza IllegalArgumentException con un mensaje claro. Lo usan cifrar y descifrar.
- **cifrar(texto, a, b):** primero llama a validarClaves. Luego recorre el texto carácter por carácter con un StringBuilder. Guarda si la letra era mayúscula, la pasa a minúscula, busca su posición con ALFABETO.indexOf y, si está en el alfabeto, aplica mod27(a × posición + b) y devuelve la letra con su mayúscula o minúscula original. Si no está en el alfabeto, copia el carácter sin cambios.
- **descifrar(texto, a, b):** llama también a validarClaves, calcula una sola vez el inverso de a antes del ciclo y aplica mod27(aInverso × (posición − b)) a cada letra. Los demás caracteres se copian igual.
- **CifradoAfinGUI:** la interfaz solo guarda una instancia de CifradoAfin (atributo logica), de modo que la lógica y la ventana están separadas. Usa dos JTextArea dentro de JScrollPane (entrada editable y resultado de solo lectura), dos JTextField para a y b y cuatro botones. El método procesarTexto lee el texto, valida que no esté vacío, convierte las claves con Integer.parseInt, llama a cifrar o descifrar y muestra los errores con JOptionPane. La ventana se inicia con SwingUtilities.invokeLater y los botones responden mediante ActionListener. Usar Resultado copia el resultado a la entrada y vacía el resultado; Limpiar vacía la entrada y el resultado y conserva las claves. Al inicio de procesarTexto se borra el resultado anterior, para no mostrarlo como actual si la operación falla.

4. Validaciones realizadas

- **Entrada vacía:** procesarTexto revisa texto.isEmpty() y muestra una advertencia (caso 6).
- **Claves no enteras o vacías:** Integer.parseInt lanza NumberFormatException y se muestra un mensaje de error (caso 8).
- **Clave a sin inverso:** si mcd(a, MODULO) es distinto de 1, validarClaves lanza IllegalArgumentException y la interfaz muestra «La clave a no es coprima de 27.» (caso 7).
- **Claves fuera de rango:** a debe estar entre 1 y 26 y b entre 0 y 26; si no, se muestra «La clave a debe estar entre 1 y 26.» o «La clave b debe estar entre 0 y 26.».
- **Resultado anterior:** procesarTexto borra el área de resultado antes de validar, así que un resultado previo no se muestra como actual si la operación falla.
- **Residuos negativos:** mod27 normaliza el resultado, como en descifrar A con a = 5 y b = 8: −88 = 20 = T (caso 9).
- **Conservación del texto:** mayúsculas, minúsculas, espacios, saltos de línea, números, signos y vocales con tilde no se alteran (casos 3, 4, 5 y 11).

5. Preguntas de análisis

¿Por qué debe conservarse la misma clave? Porque descifrar aplica la inversa exacta de la función de cifrado, $M = a^{-1} \times (C - b)$, y esa inversa solo deshace la transformación si usa los mismos a y b. Con otras claves no se produce un error: se obtiene otro texto. Por ejemplo, PCJI (HOLA cifrado con a = 5 y b = 8) se descifra como WEAP con a = 5 y b = 9 y como FDEA con a = 7 y b = 8. Por eso el algoritmo no puede reconocer una clave incorrecta.

¿Qué ocurre con los caracteres fuera del alfabeto? Los espacios, saltos de línea, números, signos y vocales con tilde se copian sin cambios, mientras que Ñ y ñ sí se cifran porque están en el alfabeto de 27 letras. Esto deja a la vista la longitud de las palabras, la puntuación y las vocales acentuadas: en «Qdkj oc kkjod Métcwvw!» se sigue viendo la é y la estructura de la frase.

¿Por qué a debe ser coprimo con el módulo? Si MCD(a, 27) ≠ 1, la función deja de ser inyectiva y no existe inverso modular. Con a = 3 y b = 8, las letras A (0), J (9) y R (18) producen la misma letra cifrada: (3 × 0 + 8) = 8, (3 × 9 + 8) = 35 mod 27 = 8 y (3 × 18 + 8) = 62 mod 27 = 8, es decir, la I. Al descifrar una I no se puede saber cuál era la letra original. En el programa, inversoModular(3) devuelve −1 y la validación rechaza la clave (caso 7). En la guía, el módulo es 26 y por eso a = 2 y a = 13 (comparten factor con 26) se rechazan; con módulo 27 los valores inválidos son los múltiplos de 3.

¿Cómo se resolvieron los residuos negativos? Con la función mod27, que calcula ((n % 27) + 27) % 27. En descifrar A con a = 5 y b = 8, la operación 11 × (0 − 8) = −88 da −7 con el operador %; al sumarle 27 se obtiene 20 y 20 % 27 = 20, la letra T.

¿Por qué este algoritmo no ofrece seguridad suficiente? Es un cifrado de sustitución monoalfabética: cada letra se reemplaza siempre por la misma, así que se conservan las frecuencias y los patrones del idioma y es vulnerable al análisis de frecuencias (Menezes et al., 1996). Además, el espacio de claves es muy pequeño: solo 18 × 27 = 486 combinaciones, que un programa puede probar en un instante. Con unos pocos pares de letra original y letra cifrada conocidos también se pueden calcular a y b. Conservar espacios, números y signos entrega más pistas. Por eso sirve para aprender el algoritmo, pero no para proteger información real.

6. Conclusiones

- **Implementación matemática.** Se programaron a mano el algoritmo de Euclides para el MCD, el inverso modular por búsqueda del 1 al 26 y la normalización de residuos con mod27, sin javax.crypto ni BigInteger. Con el alfabeto de 27 letras (a–z y ñ) la clave a debe cumplir MCD(a, 27) = 1, lo que deja 18 valores válidos, y el cálculo manual de HOLA (PCJI) coincidió con la salida del programa.
- **Funcionamiento de la aplicación.** La interfaz Swing, separada de la lógica, permite cifrar, descifrar, reutilizar el resultado y limpiar los campos. Las pruebas mostraron que conserva mayúsculas, minúsculas, saltos de línea, signos y vocales con tilde, que recupera el texto original al descifrar con las mismas claves y que informa los errores (entrada vacía, clave no entera y clave sin inverso) sin cerrar la ventana.
- **Limitaciones del mecanismo.** El cifrado afín solo tiene 486 claves válidas con 27 letras, mantiene las frecuencias de las letras y deja sin cifrar espacios, números, signos y tildes. Un cambio de clave no se detecta porque siempre produce un texto, aunque sea incorrecto. Por estas razones es útil como ejercicio académico, pero no protege datos reales.

7. Fuentes y apoyos utilizados

Guía de la práctica (rúbrica, formulación matemática y casos de prueba); Menezes et al. (1996) para la descripción de los cifrados clásicos y su criptoanálisis; y la documentación de Oracle (s. f.-a, s. f.-b) para JTextArea y el hilo de despacho de eventos de Swing. Las referencias completas en APA 7 están en «Referencias de apoyo».

Entregables y ejecución

Entregue un archivo ZIP denominado Apellido_Nombre_CifradoAfin.zip. Debe contener:

1. Proyecto Java completo, con todos los archivos .java necesarios y comentarios que expliquen las decisiones importantes.
2. Informe en PDF con cálculos, explicación, evidencias, tabla de pruebas, conclusiones y referencias.
3. Archivo README con versión de JDK, clase principal e instrucciones para compilar y ejecutar.
4. Si trabaja en equipo por indicación del docente, identificación del aporte de cada integrante.

La entrega debe poder compilarse sin dependencias criptográficas. El proyecto puede organizarse en dos clases principales: CifradoAfin para la lógica y CifradoAfinGUI para la ventana. Se acepta otra organización si mantiene la separación y la explica.

Criterios de evaluación

Criterio	Evidencia de cumplimiento	Puntos
Lógica de cifrado y descifrado	Implementación propia, conservación del texto y recuperación correcta.	3.0
Comprensión matemática	Cálculo manual, MCD, inverso y módulo explicados correctamente.	2.0
Interfaz y eventos	Ingreso y salida claros, controles y cuatro botones funcionales.	1.5
Validaciones	Manejo de entradas vacías, claves incorrectas y residuos negativos.	1.5
Pruebas y documentación	Casos ejecutados, evidencias, README y conclusiones.	1.0
Demostración y defensa	Explicación del código y resolución de un ejemplo durante la revisión.	1.0
Total		10.0

Preguntas orientadoras para la defensa

¿Cómo transforma una letra en un número y viceversa? ¿Por qué a=2 impide descifrar de forma única? ¿Cómo encontró el inverso de 5? ¿Qué hace su función con un número negativo? ¿Qué expone conservar números y signos sin cifrar? ¿Cómo comprobaría que una clave distinta no garantiza recuperar el mensaje original?

Referencias de apoyo

Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). Handbook of applied cryptography. CRC Press. https://cacr.uwaterloo.ca/hac/

Oracle. (s. f.-a). JTextArea. Java SE 17 & JDK 17. https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/javax/swing/JTextArea.html

Oracle. (s. f.-b). The event dispatch thread. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/concurrency/dispatch.html